

Datamodellsbeslut #356 — Uppdatering & Jämförelse

Datum: 2026-05-06

Ny källa: `session_2026-05-06T07-57.json` (rösttranskription, Mats)

Jämförs mot: `datamodel_beslut_356.md` (2026-05-05)

Innehåll

- 1. Bekräftelse — vad som stämmer med min tidigare analys
- 2. Nya insikter — information som inte fanns i förra sessionen
- 3. Korrigeringar — om något i min tidigare analys var fel
- 4. Uppdaterad rekommendation — om något behöver justeras

1. Bekräftelse

Den nya sessionen bekräftar i stort sett hela den tidigare analysen. Mats upprepar och förtydligar samma grundmodell som låg till grund för Alt B-rekommendationen.

1.1 OrderConcept → N Tasks — bekräftat ✓

Mats beskriver ordagrant:

“En order ifrån en kund manifesteras i systemet genom då ett orderkoncept, där man då specificerar ett visst antal artiklar i form av tjänster eller fysiska delar.”

“Om du har hundra stycken matavfallskärl i det här klustret så kommer den här tjänsten att haka fast på dem då och multipla sig själv. Så du får då hundra stycken uppdrag.”

Exakt match med tidigare analys: Orderkoncept fungerar som mall/specifikation som expanderas till N uppgifter (`work_orders`). Multiplikeringen sker mot objekt i ett kluster. Vår `expandConceptToTasks()` -funktion i Alt B modellerar precis detta flöde.

1.2 Artiklar = tjänster + fysiska delar — bekräftat ✓

Mats nämner:

- **Tjänster:** tvätt av matavfallskärl, byte av lucka
- **Fysiska delar:** klistermärke, lucka

Artiklar identifieras med artikelnummer i kundens/säljarens databas med tillhörande priser. Stämmer med befintlig `articles` -tabell och `work_order_lines` -modellen.

1.3 executionCode / utförandetyt på artikeln — bekräftat ✓

“På artikeln [finns] en ytterligare association som vi kallar då utförandetyt. Så att om vi nu har en tvätt [...] av matavfallskärl, så kommer den här listan den här att kunna sorteras då så att vi kan styra den till rätt utförare.”

“Rullbara kärl [vs] bottenöppnade avfallsbehållare [...] har en annan utförandetyt.”

Bekräftar: `executionCode` hör hemma på artikeln (source of truth), inte på uppgiften. Uppgiften ärver sin utförandetyt från artikeln vid expansion. Stämmer exakt med vår rekommendation: `articles.execution_code` som primärt fält, `work_orders.executionCode` som cachat/ärvt värde.

1.4 Kluster → Objekt → Association-matchning — bekräftat ✓

“Orderkonceptet [...] styras in mot det här klustret. Och om du då har en tjänst som heter tvätta matavfallskärl så kommer den då leta objekt som har metadata som associerar då till den här tjänsten.”

Stämmer med vår modell: `order_concepts.cluster_id` pekar på ett kluster, `order_concept_lines.association_type` bestämmer vilka objekttyper artikeln matchas mot.

1.5 Objekt-metadata och associationer — bekräftat ✓

Mats bekräftar att objekt har metadata, inklusive artikelassociationer. Detta styrker att befintlig `objects`-tabell med metadata/associations är tillräcklig för matchningslogiken i `expandConceptToTasks()`.

1.6 Informationsarv till uppgifter — bekräftat ✓

“Adressen och bakomliggande information [...] nycklar hur man kommer in och avisering [...] kommer då också hakas på för den här ordern baserat på orderkonceptet.”

Bekräftar att uppgiften ärver kontextuell information (adress, åtkomstinstruktioner, aviseringstillningar) från objektet och orderkonceptet. Stämmer med befintlig `work_orders`-struktur (`taskLatitude/taskLongitude` , `what3words` , etc.) som fylls vid expansion.

1.7 Prishantering per uppgift — bekräftat ✓

“Det står pris och så vidare då.”

Varje genererad uppgift har pris. Stämmer med `work_order_lines.resolvedPrice` som sätts vid expansion.

2. Nya insikter

2.1 Administrativa uppgiftstyper — NY DETALJ NEW

Mats introducerar en kategori som inte var explicit i förra sessionen:

“Det kan ju också vara uppgifter som är att kunden ska förhandsaviseras. Ett samtal eller vi ska kolla en kod för att komma in. Och det är ju då en uppgiftstyp som kanske vare sig behöver kranbil eller någon annan bil, utan det är en ren administrativ uppgift.”

Konsekvens för datamodellen:

- Administrativa uppgifter (telefonsamtal, kodkontroll) genereras som `work_orders` precis som fälttjän-

ster

- De har en egen `executionCode` /utförandetyp som inte kräver fordon
- De behöver kunna tilldelas separat till administrativ personal (inte fältpersonal)
- **Påverkan Alt B:** Modellen hanterar detta redan — en artikel med `execution_code = 'admin'` (eller liknande) skapar `work_orders` som tilldelas administrativa resurser. Ingen schemaändring krävs, men `expandConceptToTasks()` måste hantera att administrativa uppgifter inte nödvändigtvis matchas mot fysiska objekt.

Viktig fråga: Ska administrativa uppgifter ha `objectId`? Nuvarande schema har `objectId NOT NULL` på `work_orders`. Administrativa uppgifter (ring kund, kolla kod) kopplas till kund/kluster snarare än ett specifikt objekt. **Potentiell schemakonflikt** — kan kräva att `objectId` görs nullable eller att ett "virtuellt" administrativt objekt används.

2.2 Anmälsättare (dispatcher-roll) — NY ROLLBESKRIVNING NEW

"Det blir då upp till anmälsättaren att välja det här då."

En **anmälsättare** (dispatcher) avgör vilken utförare som tilldelas uppgifter. Detta är mer explicit än i förra sessionen och stärker behovet av ett planerings-UI (Fas 3) med stöd för:

- Filtrering/sortering efter utförandetyp
- Manuell tilldelning av uppgifter till utförare
- Vy som separerar uppgiftstyper (fält vs admin)

2.3 Utrustningskrav som sorteringsdimension — FÖRTYDLIGAD NEW

"Vissa behöver en kranbil. I vissa fall så behöver du då någon annan typ av bil för att tvätta det."

Tidigare visste vi att `executionCode` styr matchning mot resurser. Nu framgår att utrustningskrav (kranbil, tvättbil) är den primära sorteringsdimensionen. Stärker att `resources.executionCodes` (befintlig array) + `articles.execution_code` (ny) är rätt modell.

2.4 Slottider / tidsfönster — FÖRTYDLIGAD NEW

"En livsmedelsbutik som inte vill att det ska komma klockan sju på kvällen [...] Så där kan vi ju liksom ha en slottid att de tycker att vi ska komma [...] klockan nio fram till klockan tre."

Tidsfönster (slottider) kopplas till **kunden/objektet**, inte till orderkonceptet. Planeraren guidas av denna bakgrundsinformation vid schemaläggning.

Konsekvens: Slottider/preferenser bör ligga på objekt- eller kundnivå (snarare än på orderkonceptet). Befintlig `work_orders`-struktur har redan `plannedWindowStart` / `plannedWindowEnd` på uppgiftsnivå, samt `desiredDeliveryStart` / `desiredDeliveryEnd` som kan komma från konceptet. Slottiderna verkar vara en separat kundpreferens som styr planeringen — bör undersökas om `objects` eller `customers` behöver fält för detta.

2.5 Planeringsoptimering med tre perspektiv — NY INSIKT NEW

"Vi kan både optimera det utifrån leverantörens och kundens och inte minst så utförarens perspektiv."

Mats beskriver tre separata optimeringsperspektiv:

1. **Leverantören** (den som säljer tjänsten) — kostnadseffektivitet
2. **Kunden** — önskade tider, tillgänglighet
3. **Utföraren** — kapacitet, utrustning, rutt

Stärker behovet av att AI-planering/VRP tar hänsyn till constraints från alla tre parter. Inte en schemaändring, men en viktig arkitekturinsikt för Fas 4.

3. Korrigeringar

3.1 Inga direkta felaktigheter i tidigare analys

Den nya sessionen motsäger **ingenting** i den tidigare analysen. Alla grundantaganden visar sig korrekta:

- Alt B-strukturen (order_concepts → work_orders) matchar exakt Mats' beskrivning
- executionCode-placeringen stämmer
- Kluster-matchningslogiken stämmer
- Periodicitet nämns inte explicit i nya sessionen, men motsägs inte heller

3.2 Potentiell korrigering: objectId NOT NULL

Tidigare analys förutsätter att alla work_orders har ett objekt (`objectId NOT NULL`). Den nya sessionens beskrivning av administrativa uppgifter (ring kund, kolla kod) antyder att **inte alla uppgifter** har ett fysiskt objekt.

Korrigering: Utvärdera om `work_orders.objectId` ska bli nullable, alternativt om administrativa uppgifter representeras annorlunda (t.ex. via en "admin-objekt"-abstraktion eller en separat uppgiftstyp).

3.3 Periodicitet ej bekräftad i denna session

Förra sessionen tog tydligt upp periodicitet/återkommande jobb. Den nya sessionen nämner det **inte**. Detta är ingen korrigering — periodicitetskravet kvarstår — men det innebär att vi inte har ytterligare bekräftelse. Periodicitets-fälten i `order_concepts` (`is_recurring`, `recurrence_interval`, etc.) baseras fortfarande enbart på den första sessionen.

4. Uppdaterad rekommendation

Grundrekommendation: Alt B kvarstår

Den nya sessionen **stärker** Alt B-rekommendationen. Inget talar för att byta till Alt A eller Alt C.

Kompletteringar till Alt B baserat på nya insikter

4.1 Hantera administrativa uppgifter

Ny övervägan: Lägg till `task_category` (eller liknande) på `order_concept_lines` och/eller `work_orders` :

```
-- På order_concept_lines (mall-nivå)
ALTER TABLE order_concept_lines ADD COLUMN task_category TEXT DEFAULT 'field';
-- Värderna: 'field' (fältuppgift), 'admin' (administrativ), 'logistics' (material)

-- Alternativt: Cachat på work_orders vid expansion
ALTER TABLE work_orders ADD COLUMN task_category TEXT DEFAULT 'field';
```

Alternativt kan detta lösas helt via `executionCode` — en administrativ artikel har en `executionCode` som matchar administrativa resurser (utan fordonskrav).

Rekommendation: Lös via `executionCode` i första hand (enklast, inget nytt fält). Utvärdera `task_category` om det behövs för filtrering i planerings-UI.

4.2 Utvärdera `objectId`-constraint

Ny åtgärd: Undersök om `work_orders.objectId NOT NULL` ska relaxeras till nullable. Om administrativa uppgifter (ring kund för avisering, kolla kod) inte har ett fysiskt objekt, behöver de kunna skapas utan `objectId`.

Alternativ: Skapa ett "virtuellt" administrativt objekt per kluster/kund som representerar administrativa uppgifter. Behåller NOT NULL-constrainten men kräver konvention.

Rekommendation: Gör `objectId` nullable. Det är renare än att skapa phantom-objekt och enklare att hantera downstream. Befintlig kod som förutsätter `objectId` behöver granskas men påverkas minimalt (adminstuff filtreras ändå bort från VRP/rutt).

4.3 Kundpreferenser / slottider

Ny åtgärd: Kartlägg var slottider/kundpreferenser bör lagras. Kandidater:

Alternativ	Beskrivning	Bedömning
På <code>customers</code>	Kundens generella tidspreferenser	Fungerar om kunden har enhetliga preferenser
På <code>objects</code>	Per-anläggningspreferenser	Mer flexibelt (butik A vs butik B)
Separat tabell <code>delivery_preferences</code>	Full flexibilitet med historik	Mest flexibelt men mer komplexitet

Rekommendation: Starta med fält på `objects` eller `customers` (JSONB `delivery_preferences`). Utred i Fas 3 UI-design.

Uppdaterat ROM-estimat

Grundestimatet på ~23 dagar kvarstår. De nya insikterna påverkar främst **Fas 3** (planerings-UI) med:

- +0.5 dag: Hantering av administrativa vs fältuppgifter i UI
- +0.5 dag: Utvärdering och eventuell ändring av `objectId`-constraint

Nytt total-estimat: ~24 dagar (marginell ökning).

Sammanfattning av ändringar

Aspekt	Tidigare analys	Uppdatering
Alt B-rekommendation	★ Rekommenderad	★ Förstärkt — nya session-en bekräftar alla grundantaganden
executionCode på artikel	Bekräftat	Ytterligare bekräftat med konkreta exempel (kranbil vs tvättbil)
Expansion koncept → tasks	Bekräftat	Ytterligare bekräftat (100 matavfallskärl-exemplet)
Administrativa uppgifter	Ej explicit hanterat	 Nytt krav — behöver stöd för uppgifter utan fysiskt objekt
objectId NOT NULL	Förutsatt	⚠ Bör utvärderas — kan behöva bli nullable
Dispatcher-roll (anmälsättare)	Implicit i planeringsbehov	 Explicit bekräftad — stärker Fas 3 UI-behov
Slottider/kundpreferenser	Ej i scope	 Ny insikt — behöver kartläggas
Tre optimeringsperspektiv	Delvis känt	 Förtydligat — leverantör + kund + utförare
ROM-estimat	~23 dagar	~24 dagar (marginell ökning)
Periodicitet	Bekräftat (session 1)	Ej nämnt i session 2 (kvarstår oförändrat)

Appendix: Sessionsjämförelse

Aspekt	Session 1 (2026-05-05)	Session 2 (2026-05-06)
Fokus	Tekniska beslut, alternativanalys	Grundlogik, affärsflöde
Djup	Detaljerat kring datamodell	Konceptuellt kring hur systemet fungerar
Nya detaljer	Periodicitet, executionCode-placering	Administrativa uppgifter, dispatcher-roll, slottider
Format	Diskussion med tekniska avvägningar	Mats förklarar grundflödet (pedagogiskt, upprepande)
Beslut	Alt A/B/C-analys	Inget nytt beslut — bekräftar befintlig riktning

Notering om sessionens karaktär

Session 2 verkar vara en **upprepning och genomgång** av grundlogiken snarare än ett beslutsamtal. Mats förklarar flödet steg för steg, troligen för att briefa Mats (utvecklaren) eller dokumentera systemlogiken. Sessionen innehåller inga explicita beslut om datamodellval men ger värdefulla affärslogik-detaljer som stärker och kompletterar den tekniska analysen.